

Problem Set 12

The Great Spreeland Logistics Sync

Nils Schön

Angewandte Modellierung und Systemsimulation

Sommersemester 2026

1 Protocol Architecture Design

Für die Lieferkette von Burg/Spreewald nach Cottbus werden die sechs Protokolle als komplementäre Schichten eingesetzt. MCP verbindet den Dispatcher mit Werkzeugen und Daten, A2A mit fremden Agenten, UCP mit dem Handelsprozess und AP2 mit der Zahlungsautorisierung. A2UI beschreibt die Benutzeroberfläche; AG-UI transportiert laufende Ereignisse zwischen Agent und Frontend.

Interaktion	Protokoll	Aufgabe
Brückendaten	MCP	Tool-Discovery und Datenabfrage
Wetterexperte	A2A	Agent-Discovery und Delegation
Beschaffung	UCP + AP2	Checkout und Autorisierung
Live-Dashboard	A2UI + AG-UI	UI-Schema und Streaming

1.1 1. Infrastructure Discovery: MCP

Der Dispatch Agent greift nicht direkt auf die PostgreSQL-Datenbank der Stadt zu. Stattdessen veröffentlicht ein MCP-Server ein klar begrenztes Werkzeug wie `get_bridge_status`. Nach dem Verbindungsaufbau fragt der Client mit `tools/list` die verfügbaren Werkzeuge ab und ruft das passende Werkzeug anschließend mit `tools/call` auf. Die Antwort enthält Brückenstatus, Traglast, Wartungsfenster und Beobachtungszeit. Dieses Vorgehen entspricht der Discovery- und Aufruftrennung der MCP-Toolspezifikation [1].

Für den realen Betrieb sollte der MCP-Server einen read-only Datenbankbenutzer und parametrisierte Abfragen verwenden. Zugangsdaten dürfen nicht im Prompt oder Quelltext stehen. In der beigefügten Demonstration wird die nicht zugängliche städtische Datenbank durch drei lokale Datensätze ersetzt. Tool-Discovery und MCP-Aufruf werden jedoch tatsächlich über einen stdio-MCP-Prozess ausgeführt.

1.2 2. Expert Consultation: A2A

Der Weather-Predictor stellt eine A2A Agent Card bereit. Sie beschreibt Name, Endpunkt, Protokollversion, unterstützte Ein- und Ausgabeformate sowie Skills. Der Dispatcher sucht nach einem Skill mit dem Tag `route-weather`, prüft die Streaming-Fähigkeit und sendet Region, Route und Lieferfenster an das kompatible Interface. Dadurch kann ein Agent eines anderen Teams verwendet werden, ohne dessen interne Implementierung kennen zu müssen.

Die A2A-Spezifikation verwendet Agent Cards genau für diese Fähigkeitsbeschreibung und definiert Nachrichten, Tasks und optionale Streaming-Updates [2]. Die Python-Demo enthält eine

Wetterkarte und zwei Lieferantenkarten im A2A-1.0-Format. Da keine externen A2A-Server bereitgestellt wurden, beantwortet ein lokaler Adapter die Nachrichten deterministisch.

1.3 3. Secure Fulfillment: UCP und AP2

UCP und AP2 erfüllen verschiedene Aufgaben. UCP standardisiert Produkt, Warenkorb und Checkout. Der Lieferant veröffentlicht ein UCP-Profil; der Dispatcher erzeugt eine Checkout-Session für exakt 2000 kg Bio-Gurken und erhält Preis, Fracht, Lieferbedingungen und die finale Bestellung [3].

AP2 ergänzt den Checkout um kryptographisch prüfbare Autorisierung. Der Eigentümer bestätigt auf einer vertrauenswürdigen Oberfläche Händler, Ware, Menge, Höchstbetrag, Währung und Gültigkeitszeit. Ein Checkout Mandate wird an den finalen, vom Händler bestätigten Checkout gebunden. Das Payment Mandate autorisiert anschließend den konkreten Betrag und das Zahlungsmittel. Händler und Zahlungsdienstleister prüfen Signaturen, Ablaufzeit und Checkout-Bindung. Dadurch ist belegbar, dass der Agent nur den freigegebenen Kauf ausgeführt hat [4].

Die lokale Demonstration prüft ein Budget von 4000 EUR sowie Warenkorb-Hash und Manipulationsschutz. Sie ist ausdrücklich als Kursdemo markiert und nicht als echte AP2-Zahlung. Eine reale Bestellung dürfte erst nach einer Eigentümersignatur über eine AP2 Trusted Surface abgeschlossen werden.

1.4 4. Dynamic Visualization: A2UI und AG-UI

Die Lieferansicht wird nicht als generierter React- oder Flutter-Code übertragen. Der Agent sendet eine deklarative A2UI-Nachricht mit einer **Card**, einer **Column**, mehreren **Text**-Komponenten und einem getrennten Datenmodell. Der Client bildet diese bekannten Komponenten auf seine nativen Widgets ab. Das Schema kann deshalb in verschiedenen Frontends gerendert werden und enthält keinen beliebigen ausführbaren Code [5].

AG-UI übernimmt den Ereignisstrom zwischen Agenten-Backend und Frontend. Der Dispatcher sendet Start-, Fortschritts-, Zustands- und Abschlussereignisse. Die A2UI-Nachricht wird dabei als strukturiertes Custom Event transportiert:

```
RUN_STARTED
STEP_STARTED -> TEXT_MESSAGE_CONTENT -> STEP_FINISHED
STATE_SNAPSHOT
CUSTOM(name="a2ui", value=<delivery card>)
RUN_FINISHED
```

Damit sieht der Benutzer während der Verarbeitung, ob Brückendaten, Wetterabfrage und Lieferantenverhandlung abgeschlossen wurden. AG-UI standardisiert hierfür einen transportunabhängigen, ereignisbasierten Ablauf [6].

2 Umsetzung des Dispatcher Swarms

Der Google-ADK-Agent bindet den lokalen Infrastrukturserver als **McpToolset** ein. Der ausführbare Offline-Lauf benötigt keinen Cloud-Key und führt folgende Schritte aus:

1. Entdeckung der MCP-Werkzeuge und Abfrage aller Brücken,
2. Discovery des Wetteragenten über seine Agent Card,
3. Auswahl einer offenen und wettergeeigneten Route,

4. Angebotsabfrage bei zwei Lieferantenagenten,
5. Budgetprüfung und Ausgabe der A2UI-Karte über AG-UI.

Im Beispieldatensatz wird die offene Brücke SPB-014 auf der Route über Lübbenau gewählt. Bio-Spree bietet 2 000 kg Bio-Gurken einschließlich Fracht für 3 790 EUR an; das zweite gültige Angebot liegt bei 3 860 EUR. Daher wählt der Dispatcher Bio-Spree nach dem niedrigsten Gesamtpreis.

Der Lauf kann im Projektverzeichnis reproduziert werden:

```
uv sync
uv run python run_spreeland_dispatcher.py
uv run python run_spreeland_dispatcher.py --format ag-ui
```

Die geforderten Abgabeartefakte sind damit das vorliegende Design-Dokument, der Python-Quellcode und die Datei `a2ui_delivery_card.json`.

Quellen

- [1] Model Context Protocol: *Tools*.
<https://modelcontextprotocol.io/specification/draft/server/tools>
- [2] A2A Protocol Working Group: *Agent2Agent Protocol Specification 1.0*.
<https://a2a-protocol.org/latest/specification/>
- [3] Universal Commerce Protocol: *Official Specification – Overview*.
<https://ucp.dev/latest/specification/overview/>
- [4] Agent Payments Protocol: *AP2 Specification*.
<https://ap2-protocol.org/ap2/specification/>
- [5] A2UI Project: *A2UI Protocol v1.0 Candidate*.
https://github.com/a2ui-project/a2ui/blob/main/specification/v1_0/docs/a2ui_protocol.md
- [6] AG-UI: *Core Architecture and Events*.
<https://docs.ag-ui.com/concepts/architecture>